TRƯỜNG ĐẠI HỌC CÔNG NGHỆ
ĐẠI HỌC QUỐC GIA HÀ NỘI




# BÁO CÁO THỰC HÀNH

## MÔN: TƯƠNG TÁC NGƯỜI - ROBOT RBE3053

## BÀI THỰC HÀNH SỐ 5

TurtleBot 4: Localization, Nav2 và nhận diện vật thể YOLO trong Gazebo

**Sinh viên thực hiện:** **Lê Trọng Nghĩa – 23020754**

**Giảng viên phụ trách:** **PGS.TS. Hoàng Văn Xiêm**
**CN. Nguyễn Quốc Bảo**

# MỤC LỤC

**1. Cấu trúc chương trình . . . . . 2**

1.1. Môi trường và tổ chức package . . . . . 2

1.2. Sơ đồ khối . . . . . 2

1.3. Trình tự khởi động . . . . . 3

1.4. Cây toạ độ . . . . . 4

1.5. Các quyết định thiết kế đáng chú ý . . . . . 5

**2. Các node và package chính . . . . . 5**

**3. Các topic và action chính . . . . . 6**

**4. Khởi chạy mô phỏng, nạp bản đồ và định vị (Phần 1) . . . . . 7**

4.1. World maze và các vật thể thêm vào . . . . . 7

4.2. Nạp bản đồ và AMCL . . . . . 7

4.3. Kiểm tra định vị . . . . . 8

**5. Khảo sát toạ độ mục tiêu (Phần 2) . . . . . 9**

**6. Điều hướng tự động bằng Nav2 (Phần 3) . . . . . 10**

6.1. Kiến trúc Nav2 . . . . . 10

6.2. Costmap . . . . . 10

6.3. Lập kế hoạch và điều khiển . . . . . 10

6.4. Node điều hướng waypoint_navigator.py . . . . . 11

6.5. Kết quả điều hướng . . . . . 12

**7. Nhận diện vật thể bằng YOLO (Phần 4) . . . . . 13**

7.1. Thiết kế node . . . . . 13

7.2. Mô hình và tham số . . . . . 14

7.3. Kết quả nhận diện . . . . . 14

**8. Nhận xét và hạn chế . . . . . 16**

**9. Hình ảnh minh hoạ kết quả . . . . . 16**

**10. Liên kết video demo và mã nguồn . . . . . 17**

# 1. Cấu trúc chương trình

## 1.1. Môi trường và tổ chức package

Bài thực hành được thực hiện trên Ubuntu 22.04, ROS 2 Humble và Ignition Gazebo Fortress, chạy trên cùng máy Acer Nitro AN515-57 (12 luồng CPU, 15 GB RAM, đồ hoạ Intel UHD qua Mesa) như bài trước. Robot là TurtleBot 4 Standard gồm đế iRobot Create 3, LiDAR RPLIDAR 2D và camera OAK-D, đặt trong world `maze` kích thước khoảng 20 m × 20 m. Bản đồ dùng lại của tuần trước (`maps/map.yaml`, `map.pgm`): lưới 404 × 403 ô, độ phân giải 0,05 m/ô, gốc toạ độ (−10; −10,1) m.

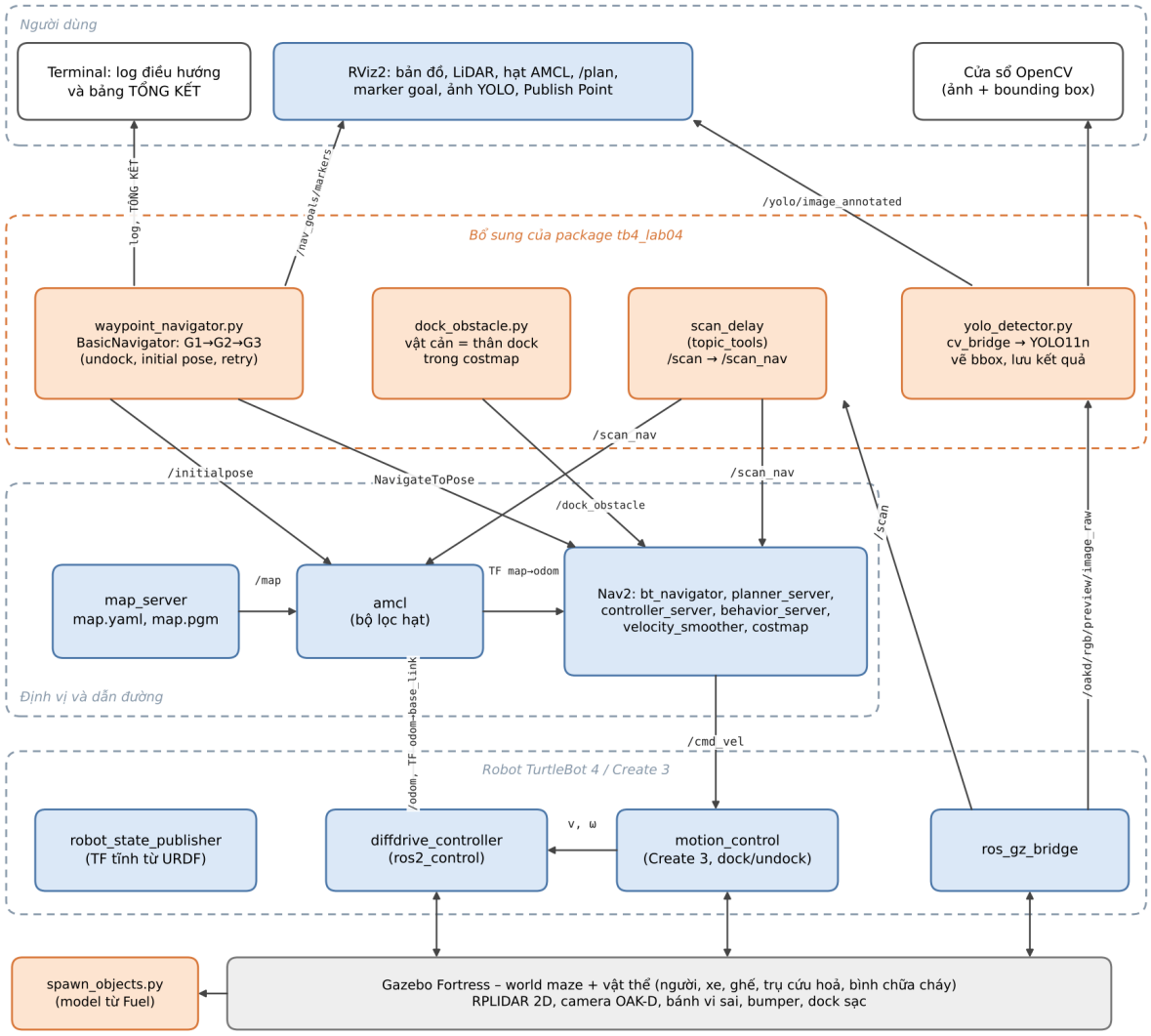
Chương trình được tổ chức thành package `tb4_lab04` (ament_cmake), kế thừa package của bài trước. Package không viết lại mô phỏng hay Nav2 mà dùng các launch chính thức của TurtleBot 4 và Navigation2; phần bổ sung gồm hai node ứng dụng (điều hướng qua danh sách goal và nhận diện YOLO), script thêm vật thể vào mô phỏng và các công cụ kiểm tra (Bảng 1).

| Thư mục / file | Nội dung |
|---|---|
| `launch/navigation.launch.py` | Phần 1: Gazebo, TurtleBot 4, thêm vật thể, nạp bản đồ, AMCL, Nav2, RViz2 |
| `launch/yolo.launch.py` | node YOLO chạy riêng (tham số yolo_model, conf, device, show_window) |
| `launch/demo.launch.py` | tất cả trong một lệnh: navigation + YOLO + node điều hướng |
| `scripts/waypoint_navigator.py` | node điều hướng tự động G1 → G2 → G3 (nav2_simple_commander) |
| `scripts/yolo_detector.py` | node nhận diện vật thể (cv_bridge + ultralytics YOLO) |
| `scripts/spawn_objects.py` | tải model từ Gazebo Fuel và đặt vào world lúc chạy |
| `scripts/check_goals.py` | kiểm tra goal (cách tường, tới được, camera nhìn thấy vật thể) và vẽ ảnh bản đồ, quỹ đạo, vị trí phát hiện |
| `scripts/goal_recorder.py` | ghi toạ độ từ Publish Point (/clicked_point) hoặc /amcl_pose ra YAML |
| `scripts/localization_check.py` | đo độ khớp giữa tia LiDAR và tường của bản đồ |
| `scripts/doctor.sh` | kiểm tra môi trường, thư viện YOLO (numpy, cv_bridge, ultralytics) |
| `config/` | goals.yaml, objects.yaml, yolo.yaml, localization.yaml, nav2.yaml (và cấu hình bài trước) |
| `maps/` | map.yaml, map.pgm của tuần trước |
| `rviz/, test/` | cấu hình RViz2 (panel ảnh YOLO, marker goal); kiểm thử đơn vị cho phần không phụ thuộc ROS |

*Bảng 1 – Cấu trúc package* `tb4_lab04`.

## 1.2. Sơ đồ khối

Sơ đồ khối của hệ thống được trình bày ở Hình 1, gồm năm tầng. Tầng dưới cùng là Gazebo mô phỏng world `maze` cùng các vật thể thêm vào (người, xe, ghế, trụ cứu hoả, bình chữa cháy), LiDAR, camera OAK-D, bánh vi sai và đế sạc. Tầng robot gồm `ros_gz_bridge` chuyển dữ liệu giữa Gazebo và ROS 2, bộ điều khiển vi sai `diffdrive_controller` phát odometry, và node `motion_control` của Create 3 nhận lệnh vận tốc. Tầng định vị và dẫn đường gồm `map_server` (nạp bản đồ tuần trước), `amcl` (định vị trên bản đồ đó) và Nav2 (lập kế hoạch, điều khiển). Tầng bổ sung của package gồm node điều hướng `waypoint_navigator.py` đóng vai trò client của Nav2 và node `yolo_detector.py` đọc ảnh camera. Trên cùng là người dùng, quan sát qua RViz2, terminal và cửa sổ OpenCV; người dùng không điều khiển robot trong lúc node điều hướng chạy.



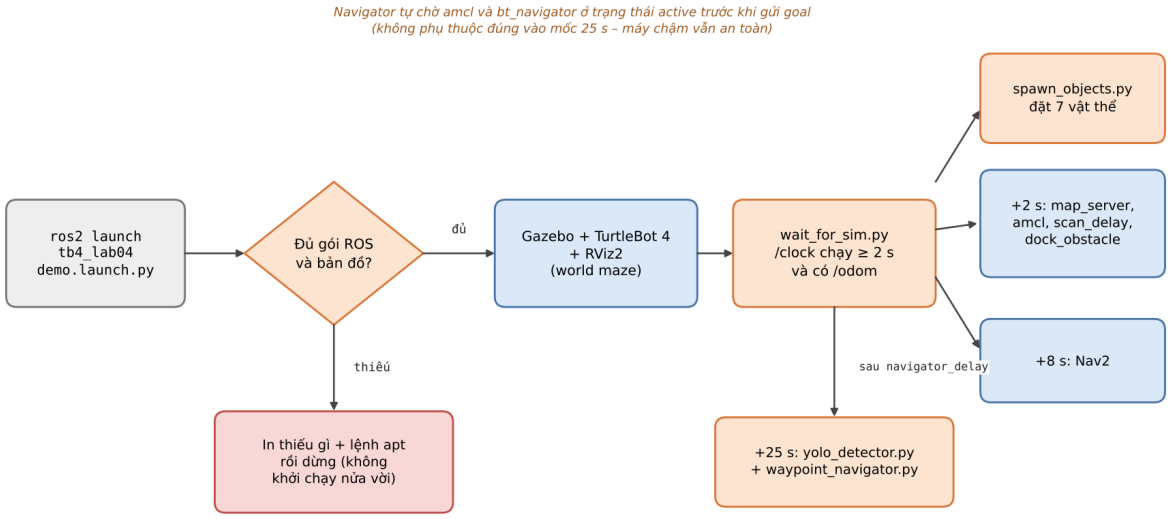

*Hình 1 – Sơ đồ khối hệ thống (xanh: gói chính thức; cam: bổ sung của package* `tb4_lab04`*).*

Hai luồng dữ liệu độc lập chạy song song. Luồng điều hướng: node điều hướng gửi action `NavigateToPose` tới `bt_navigator`; Nav2 lập đường đi, sinh `/cmd_vel` đưa tới `motion_control`. Luồng nhận diện: ảnh `/oakd/rgb/preview/image_raw` đi qua `yolo_detector.py` và trả ảnh đã vẽ khung ra cửa sổ OpenCV, `/yolo/image_annotated` (xem trên RViz2) và `/yolo/detections`. Hai luồng chỉ gặp nhau ở vị trí robot: node YOLO ghi lại vị trí AMCL của robot lúc phát hiện từng vật thể để vẽ lên bản đồ.

## 1.3. Trình tự khởi động

Nav2 và AMCL chỉ khởi động ổn định khi Gazebo đã chạy xong; khởi động cùng lúc trên máy đang tải nặng dễ làm các lifecycle node không kết nối được với nhau. Vì vậy các launch được thiết kế theo trình tự trên Hình 2. Trước tiên, launch kiểm tra gói ROS và bản đồ cần thiết; nếu thiếu thì in rõ thiếu gì và dừng, không khởi chạy nửa vời. Sau khi Gazebo, TurtleBot 4 và RViz2 khởi động, node `wait_for_sim.py` chờ đến khi đồng hồ mô phỏng `/clock` chạy ít nhất 2 s và đã có `/odom`. Khi node này thoát, `spawn_objects.py` đặt 7 vật thể vào world, `map_server`, `amcl`, `scan_delay` và `dock_obstacle` khởi động sau 2 s, Nav2 sau 8 s (hai tham số `localization_delay` và `nav2_delay`). Với `demo.launch.py`, node YOLO và node điều hướng chạy sau `navigator_delay` = 25 s; ngoài ra node điều hướng còn tự chờ `amcl` và `bt_navigator` ở trạng thái active, nên máy chậm hơn dự kiến

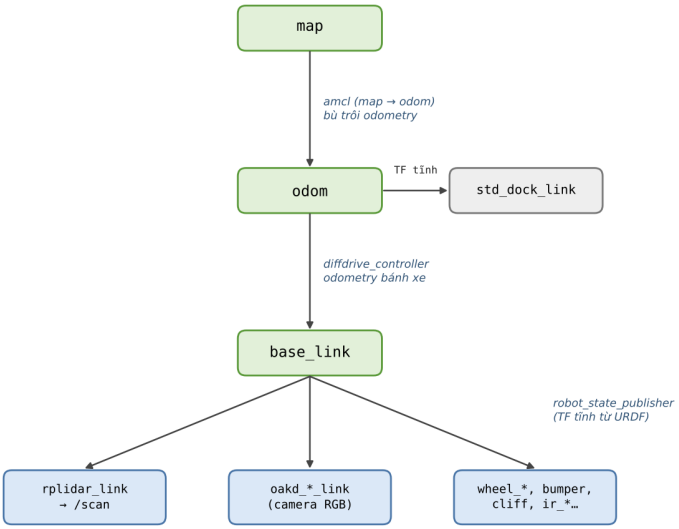
vẫn không gửi goal quá sớm.




*Hình 2 – Trình tự khởi động của* `demo.launch.py`.

## 1.4. Cây toạ độ

Cây biến đổi toạ độ (TF) được thể hiện trên Hình 3. Biến đổi `odom→base_link` do `diffdrive_controller` tính từ odometry bánh xe, liên tục nhưng trôi dần theo quãng đường. `amcl` bù sai số này bằng biến đổi `map→odom`, nhờ đó `map→base_link` là pose đã hiệu chỉnh của robot trên bản đồ. Các biến đổi từ `base_link` tới cảm biến là biến đổi tĩnh do `robot_state_publisher` phát từ mô tả URDF. Frame `map` của bản đồ tuần trước trùng với frame của world Gazebo (robot sinh ra tại (0, 0, 0) khi đang trong dock), nên toạ độ đọc từ bản đồ có thể dùng trực tiếp làm vị trí trong mô phỏng.




*Hình 3 – Cây TF và node phát từng biến đổi.*

### 1.5. Các quyết định thiết kế đáng chú ý

Phần lớn cấu hình giữ theo bản gốc của TurtleBot 4 và của bài trước. Các điểm bổ sung hoặc thay đổi trong bài này, cùng lý do, được liệt kê ở Bảng 2.

| Quyết định | Lý do |
|---|---|
| Đặt vật thể lúc chạy bằng `ros_gz_sim create`, không tạo world mới | launch của TurtleBot 4 ghi đè `IGN_GAZEBO_RESOURCE_PATH` và các topic cầu nối phụ thuộc tên world; world mới dễ làm hỏng cầu nối và không còn là đúng môi trường tuần trước. Thêm vật thể vào world `maze` gốc giữ nguyên tường và vị trí dock nên bản đồ cũ dùng lại được |
| AMCL khởi tạo tại (0, 0, 0); node điều hướng chỉ phát lại `/initialpose` khi robot còn trong dock | pose (0, 0, 0) chỉ đúng khi robot ở dock. Chạy lại giữa mê cung thì giữ ước lượng hiện có của AMCL, không ghi đè bằng pose cũ |
| Tự viết bước undock (giới hạn 60 s theo giờ mô phỏng) | có giới hạn thời gian rõ ràng, theo đồng hồ mô phỏng, và không phụ thuộc gói `turtlebot4_navigation` |
| Giữ `/scan_nav` (`/scan` trễ 0,1 s) và `dock_obstacle` của bài trước | tránh lỗi TF khi scan đến trước biến đổi tương ứng; dock thấp hơn mặt phẳng quét LiDAR nên phải đưa vào costmap để Nav2 không lập đường xuyên qua dock |
| Ghim `numpy<2` và `opencv-python<4.11` | `cv_bridge` của Humble được biên dịch với NumPy 1.x; lệnh `pip install ultralytics opencv-python` theo đề kéo NumPy 2 về và làm hỏng chuyển đổi ảnh |
| Ctrl+C huỷ goal đang chạy trước khi thoát | để robot dừng hẳn thay vì tiếp tục đi theo goal cũ; kết quả đã chạy vẫn được lưu |

*Bảng 2 – Các quyết định thiết kế và lý do.*

## 2. Các node và package chính

Các node tham gia hệ thống được liệt kê trong Bảng 3. Ngoài các gói chính thức, package `tb4_lab04` bổ sung năm thành phần chạy kèm launch. Node `wait_for_sim.py` quyết định thời điểm bật tầng định vị và Nav2. Node `dock_obstacle.py` đưa thân dock vào costmap. Node `spawn_objects.py` đặt vật thể vào Gazebo. Hai node ứng dụng chính là `waypoint_navigator.py` (Phần 3) và `yolo_detector.py` (Phần 4).

| Package | Node | Vai trò |
|---|---|---|
| `ros_gz_sim` | `ign gazebo, create` | mô phỏng vật lý và cảm biến; đặt vật thể |
| `ros_gz_bridge` | `parameter_bridge` | chuyển topic giữa Gazebo và ROS 2 (/scan, camera, /clock...) |
| `robot_state_publisher` | `robot_state_publisher` | phát TF tĩnh của robot từ URDF |
| `ros2_control` | `diffdrive_controller` | điều khiển vi sai, phát /odom và TF odom → base_link |
| `irobot_create_nodes` | `motion_control` | nhận /cmd_vel, áp giới hạn an toàn, thực hiện dock/undock |
| `nav2_map_server` | `map_server` | nạp map.yaml / map.pgm và phát /map |
| `nav2_amcl` | `amcl` | định vị trên bản đồ, phát TF map → odom và /amcl_pose |
| `nav2_bt_navigator` | `bt_navigator` | nhận action NavigateToPose, điều phối bằng cây hành vi |
| `nav2_planner` | `planner_server` | lập đường đi toàn cục (NavFn) |
| `nav2_controller` | `controller_server` | bám đường đi, sinh lệnh vận tốc (DWB) |

| Package | Node | Vai trò |
|---|---|---|
| nav2_behaviors | behavior_server | hành vi phục hồi: quay tại chỗ, lùi, chờ |
| nav2_velocity_smoother | velocity_smoother | giới hạn vận tốc và gia tốc |
| nav2_lifecycle_manager | lifecycle_manager | cấu hình và kích hoạt các node Nav2 theo thứ tự |
| topic_tools | delay (scan_delay) | tạo /scan_nav = /scan trễ 0,1 s |
| tb4_lab04 | wait_for_sim, dock_obstacle, spawn_objects | chờ mô phỏng; dock vào costmap; đặt vật thể |
| tb4_lab04 | waypoint_navigator | gửi G1, G2, G3 tới Nav2 và tổng kết kết quả |
| tb4_lab04 | yolo_detector | nhận diện vật thể từ camera, vẽ khung, lưu kết quả |
| rviz2 | rviz2 | hiển thị bản đồ, LiDAR, hạt AMCL, đường đi, ảnh YOLO; công cụ Publish Point |

*Bảng 3 – Các node và package chính.*

## 3. Các topic và action chính

Bảng 4 tóm tắt các topic và action chính. Trong Phần 1, /map do map_server phát một lần, /scan_nav vào amcl và các costmap, còn TF map→odom do AMCL phát. Trong Phần 3, node điều hướng gửi đích qua action /navigate_to_pose và nhận phản hồi gồm khoảng cách còn lại, thời gian ước tính và số lần phục hồi; Nav2 phát đường đi /plan và lệnh vận tốc /cmd_vel. Trong Phần 4, ảnh vào qua /oakd/rgb/preview/image_raw (rgb8, 320 × 240) và kết quả ra qua hai topic /yolo/*.

| Topic / action | Kiểu | Vai trò |
|---|---|---|
| /scan | sensor_msgs/LaserScan | khoảng cách từ RPLIDAR; nguồn của /scan_nav |
| /scan_nav | sensor_msgs/LaserScan | /scan trễ 0,1 s; vào AMCL và costmap |
| /odom | nav_msgs/Odometry | odometry bánh xe từ diffdrive_controller |
| /map | nav_msgs/OccupancyGrid | bản đồ do map_server phát (QoS transient local) |
| /initialpose | geometry_msgs/ PoseWithCovarianceStamped | pose ban đầu cho AMCL (node điều hướng hoặc RViz2) |
| /amcl_pose | geometry_msgs/ PoseWithCovarianceStamped | pose ước lượng của robot trong frame map |
| /navigate_to_pose | action nav2_msgs/ NavigateToPose | đích điều hướng; feedback: distance_remaining, estimated_time_remaining, number_of_recoveries |
| /plan | nav_msgs/Path | đường đi toàn cục do planner_server lập |
| /cmd_vel | geometry_msgs/Twist | lệnh vận tốc tới motion_control |
| /undock, /dock | action irobot_create_msgs/ Undock, Dock | rời và vào đế sạc |
| /dock_status | irobot_create_msgs/ DockStatus | robot có đang trong dock không (quyết định initial pose) |

| Topic / action | Kiểu | Vai trò |
|---|---|---|
| `/clicked_point` | `geometry_msgs/PointStamped` | điểm bấm bằng Publish Point trên RViz2 (khảo sát goal) |
| `/oakd/rgb/preview/image_raw` | `sensor_msgs/Image` | ảnh RGB từ camera OAK-D (đầu vào YOLO) |
| `/yolo/image_annotated` | `sensor_msgs/Image` | ảnh đã vẽ khung, xem trên RViz2 |
| `/yolo/detections` | `vision_msgs/Detection2DArray` | danh sách vật thể phát hiện (lớp, độ tự tin, khung) |
| `/nav_goals/markers` | `visualization_msgs/MarkerArray` | marker G1–G3, đổi màu theo trạng thái |

*Bảng 4 – Các topic và action chính.*

# 4. Khởi chạy mô phỏng, nạp bản đồ và định vị (Phần 1)

## 4.1. World maze và các vật thể thêm vào

Robot được khởi chạy lại trong đúng world `maze` đã dùng để quét bản đồ, tại cùng vị trí xuất phát (robot đỗ trong dock ở (0, 0, 0)). Để bài toán nhận diện có đối tượng, bảy vật thể được thêm vào world lúc chạy (Bảng 5). Model lấy từ Gazebo Fuel (cần Internet ở lần đầu, sau đó dùng bản lưu trong `~/.ignition/fuel`), đặt bằng `ros_gz_sim create`. Toạ độ vật thể nằm trong frame của world, chính là frame `map` của bản đồ. Mỗi vật thể được đặt ở khu vực khác nhau để ba goal có góc nhìn riêng.

| Vật thể | Model Fuel | x (m) | y (m) | Lớp COCO kỳ vọng |
|---|---|---|---|---|
| `car_1` | Hatchback blue | −8,70 | 7,20 | car |
| `person_1` | Standing person | −3,90 | 2,40 | person |
| `person_2` | Casual female | −6,00 | −5,60 | person |
| `chair_1` | Chair | −4,75 | −5,25 | chair |
| `fire_hydrant_1` | Fire hydrant | 8,60 | −1,00 | fire hydrant |
| `person_3` | Male visitor | 9,00 | −2,60 | person |
| `fire_extinguisher_1` | Fire Extinguisher | 8,30 | −3,30 | không có trong COCO |

*Bảng 5 – Vật thể thêm vào world* `maze`.

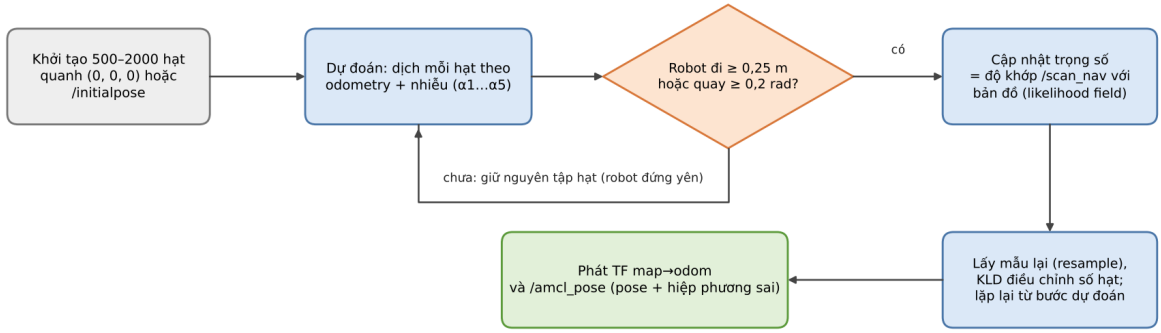
```
ros2 run tb4_lab04 spawn_objects.py --download-only   # tải model Fuel (lần đầu)
ros2 launch tb4_lab04 navigation.launch.py            # Gazebo + vật thể + AMCL + Nav2
```

## 4.2. Nạp bản đồ và AMCL

Node `map_server` nạp `maps/map.yaml` (ảnh `map.pgm`, ngưỡng `occupied_thresh` = 0,65 và `free_thresh` = 0,15) rồi phát `/map`. AMCL (Adaptive Monte Carlo Localization) ước lượng pose của robot trên bản đồ đó bằng bộ lọc hạt. Mỗi hạt là một giả thuyết pose kèm trọng số. Chu kỳ xử lý gồm ba bước (Hình 4): dự đoán, dịch mỗi hạt theo odometry cộng nhiễu; cập nhật, tính trọng số theo độ khớp giữa scan và bản đồ; lấy mẫu lại, giữ các hạt trọng số cao và loại các hạt thấp. Với mô hình `likelihood_field`, nếu tia thứ i có điểm cuối cách vật cản gần nhất trên bản đồ một khoảng d_i thì trọng số của hạt k là

$$w^{[k]} \propto \prod_{i=1}^{N} \left( z_{\text{hit}} \frac{1}{\sqrt{2\pi}\,\sigma_{\text{hit}}} e^{-d_i^2 / 2\sigma_{\text{hit}}^2} + \frac{z_{\text{rand}}}{z_{\text{max}}} \right) \tag{1}$$

Số hạt được điều chỉnh theo phương pháp KLD-sampling trong khoảng 500–2000: nhiều hạt khi còn bất định, ít hạt khi tập hạt đã hội tụ. Bộ lọc chỉ cập nhật khi robot di chuyển ít nhất 0,25 m hoặc quay ít nhất 0,2 rad nên khi robot đứng yên tập hạt không đổi. Các tham số chính nằm trong Bảng 6.




*Hình 4 – Chu kỳ xử lý của bộ lọc hạt AMCL.*

| Tham số | Giá trị | Ý nghĩa |
|---|---|---|
| `min_particles / max_particles` | `500 / 2000` | giới hạn số hạt |
| `laser_model_type` | `likelihood_field` | mô hình đo của LiDAR |
| `max_beams` | `60` | số tia dùng trong mỗi lần cập nhật |
| `alpha1 … alpha5` | `0,2` | hệ số nhiễu của mô hình chuyển động |
| `update_min_d / update_min_a` | `0,25 m / 0,2 rad` | ngưỡng chuyển động để cập nhật |
| `set_initial_pose` | `true, (0, 0, 0)` | pose khởi tạo trùng điểm sinh robot |
| `scan_topic` | `/scan_nav` | /scan trễ 0,1 s |

*Bảng 6 – Các tham số chính của AMCL (`config/localization.yaml`).*

Vì frame `map` trùng điểm sinh robot nên AMCL được khởi tạo sẵn tại (0, 0, 0) bằng `set_initial_pose`, không cần công cụ `2D Pose Estimate` trên RViz2. Công cụ này chỉ cần khi tia LiDAR hiển thị lệch khỏi tường. Node điều hướng cũng phát lại `/initialpose` (hiệp phương sai 0,25 cho x, y và 0,0685 cho yaw) khi phát hiện robot còn trong dock, và bỏ qua bước này khi robot đã rời dock để không ghi đè một ước lượng đang đúng.

## 4.3. Kiểm tra định vị

Định vị được coi là đúng khi ba điều kiện cùng thoả trên RViz2 với Fixed Frame = `map`: bản đồ hiển thị đúng; vị trí và hướng robot trên bản đồ khớp với vị trí thực trong Gazebo; các điểm quét LiDAR `/scan` trùng với các bức tường của bản đồ, còn hạt AMCL tụ quanh robot. Ngoài kiểm tra bằng mắt, script `localization_check.py` có thể đo tỉ lệ tia LiDAR có điểm cuối cách tường của bản đồ không quá 0,15 m (ba ô lưới); tỉ lệ này từ 70% trở lên thì định vị được xem là đúng.

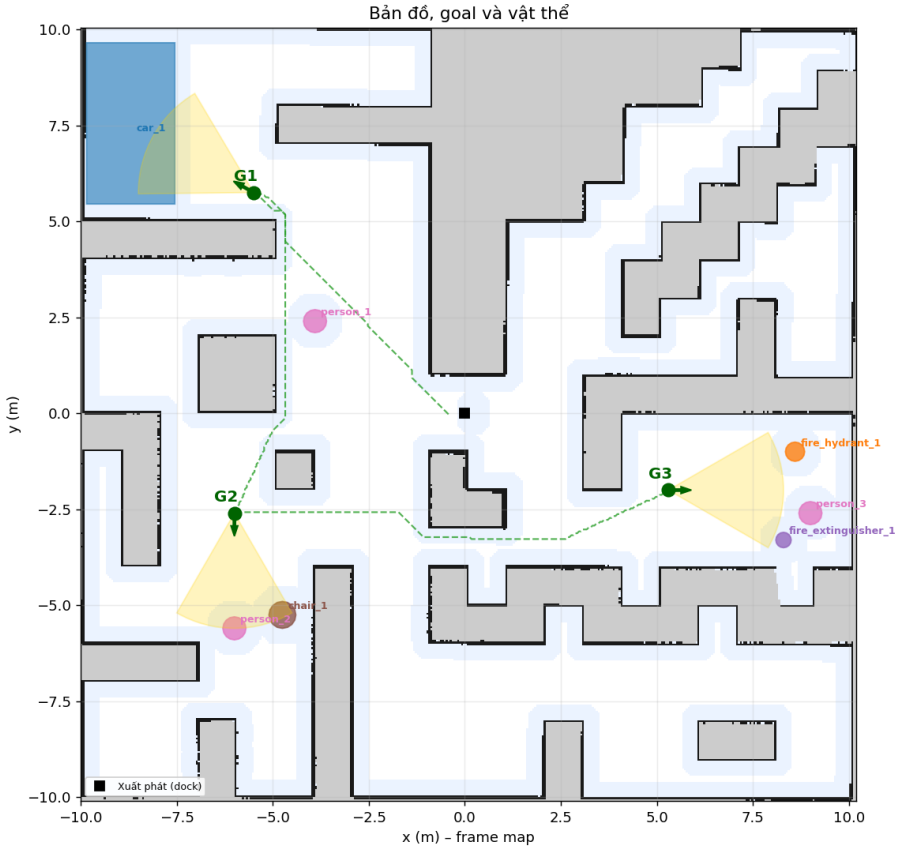
```
ros2 run tb4_lab04 localization_check.py
```

## 5. Khảo sát toạ độ mục tiêu (Phần 2)

Ba goal được đặt ở ba khu vực khác nhau của bản đồ (Hình 5), toạ độ (x, y, yaw) trong frame `map`. Các toạ độ được xác định trên bản đồ `map.pgm` và kiểm tra bằng `check_goals.py` theo ba tiêu chí: (1) cách tường ít nhất 0,6 m; (2) tới được từ điểm xuất phát, kiểm tra bằng tìm đường A* trên bản đồ đã nở theo bán kính robot 0,175 m; (3) khi tới nơi, vật thể nằm trong trường nhìn 60° của camera. Có thể đối chiếu lại bằng công cụ Publish Point trên RViz2 và đọc `/clicked_point` (script `goal_recorder.py` in sẵn đoạn YAML), hoặc đặt robot tại vị trí mong muốn rồi đọc `/amcl_pose`. Kết quả kiểm tra ở Bảng 7; yaw của G1 là 2,63 rad ≈ 151°.

| Goal | x (m) | y (m) | yaw | Cách tường | Vật thể trong tầm nhìn camera |
|---|---|---|---|---|---|
| G1 | −5,50 | 5,75 | 151° | 0,75 m | car_1 (3,5 m, lệch +5°) |
| G2 | −6,00 | −2,60 | −90° | 1,21 m | person_2 (3,0 m, +0°); chair_1 (2,9 m, +25°) |
| G3 | 5,30 | −2,00 | 0° | 1,25 m | fire_hydrant_1 (3,4 m, +17°); person_3 (3,7 m, −9°); fire_extinguisher_1 (3,3 m, −23°) |

*Bảng 7 – Toạ độ ba goal và kết quả kiểm tra (`check_goals.py`).*

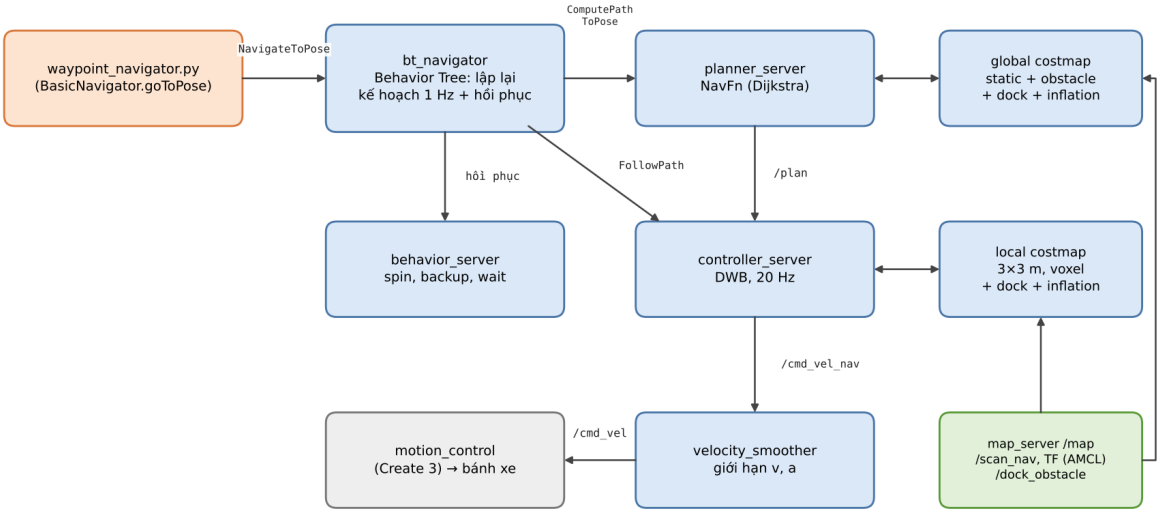



*Hình 5 – Bản đồ tuần trước, ba goal (mũi tên = hướng yaw; quạt vàng = vùng nhìn của camera 60°), vật thể (vòng tròn) và đường đi dự kiến (nét đứt xanh lá).*

# 6. Điều hướng tự động bằng Nav2 (Phần 3)

## 6.1. Kiến trúc Nav2

Khi node điều hướng gửi một đích, action `NavigateToPose` tới `bt_navigator` (Hình 6). Cây hành vi mặc định lập lại đường đi với tần số 1 Hz, chuyển đường cho bộ điều khiển và gọi các hành vi phục hồi (xoá costmap, quay tại chỗ, lùi, chờ) khi robot bị kẹt. Việc lập lại đường đi định kỳ cho phép robot thích ứng khi LiDAR phát hiện vật cản chưa có trên bản đồ, ví dụ các vật thể vừa thêm vào.




*Hình 6 – Các server của Nav2 và luồng dữ liệu tới bánh xe.*

## 6.2. Costmap

Global costmap (frame `map`) gồm lớp tĩnh từ `/map`, lớp vật cản từ `/scan_nav`, lớp dock và lớp vùng đệm (inflation). Local costmap (frame `odom`) là cửa sổ 3 × 3 m quanh robot, độ phân giải 0,06 m, cập nhật nhanh hơn để phục vụ bộ điều khiển. Robot được mô hình hoá là hình tròn bán kính 0,175 m. Lớp vùng đệm gán chi phí cho mỗi ô theo khoảng cách d tới vật cản gần nhất:

$$c(d) = \begin{cases} 253 & d \le r_{\text{robot}} \\ 252\, e^{-k\,(d - r_{\text{robot}})} & r_{\text{robot}} < d \le R \\ 0 & d > R \end{cases} \tag{2}$$

với r_robot = 0,175 m, hệ số suy giảm k = 4,0 và bán kính vùng đệm R = 0,45 m. Chi phí 253 tương ứng va chạm chắc chắn nếu tâm robot nằm tại ô đó; phần chi phí giảm dần ra phía ngoài khiến bộ lập kế hoạch ưu tiên đường ở giữa hành lang thay vì men sát tường.

## 6.3. Lập kế hoạch và điều khiển

Bộ lập kế hoạch NavFn dùng thuật toán Dijkstra trên global costmap để tìm đường có tổng chi phí nhỏ nhất từ vị trí hiện tại tới đích, kết quả phát lên `/plan`. Vì chi phí của mỗi ô gồm cả phí vùng đệm nên đường tìm được là đường cân bằng giữa độ dài và khoảng cách tới tường. Bộ điều khiển DWB chạy ở 20 Hz: tại mỗi chu kỳ lấy mẫu các cặp vận tốc (v, ω) trong giới hạn v ≤ 0,26 m/s, ω ≤ 1,0 rad/s, mô

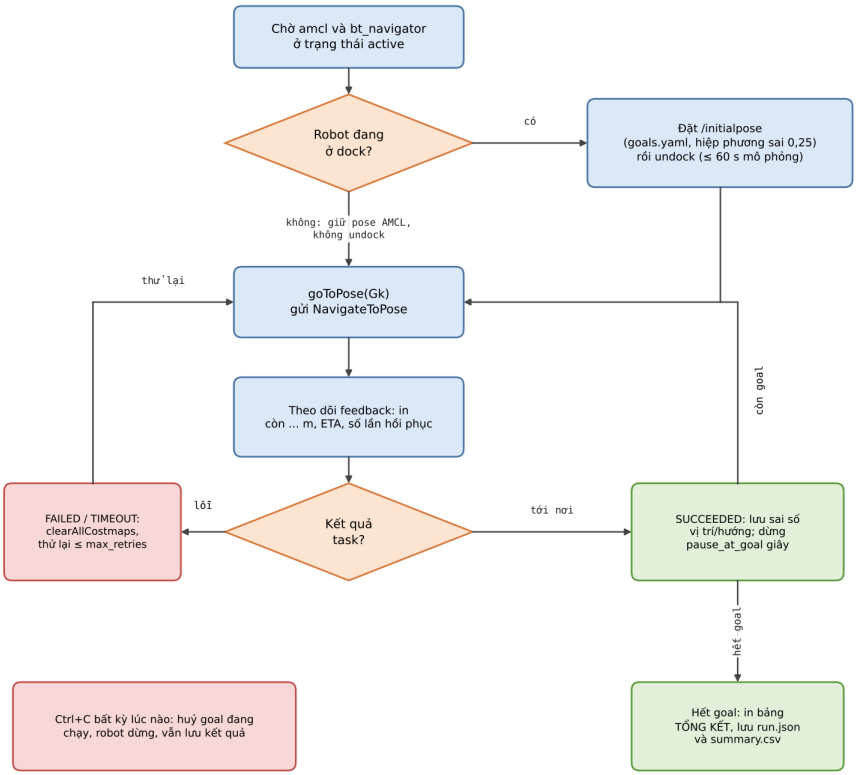
phỏng quỹ đạo tương ứng trong 1,7 s và chấm điểm bằng các hàm chi phí (bám đường đi, hướng về đích, tránh vật cản trên local costmap). Lệnh vận tốc sau đó qua `velocity_smoother` rồi tới `motion_control` của Create 3. Một goal được xem là đạt khi sai số vị trí không quá 0,25 m và sai số hướng không quá 0,25 rad:

$$\sqrt{(x - x_g)^2 + (y - y_g)^2} \le 0{,}25 \text{ m}, \qquad |\text{wrap}(\theta - \theta_g)| \le 0{,}25 \text{ rad} \tag{3}$$

## 6.4. Node điều hướng waypoint_navigator.py

Thay vì kéo thả Nav2 Goal bằng chuột, node `waypoint_navigator.py` kế thừa `BasicNavigator` của `nav2_simple_commander`, đọc danh sách goal từ `config/goals.yaml` rồi gửi tuần tự G1 → G2 → G3. Luồng xử lý (Hình 7) gồm các bước sau.

Bước 1, chờ `amcl` và `bt_navigator` ở trạng thái active (kiểm tra qua dịch vụ lifecycle) thay vì chờ một khoảng thời gian cố định. Bước 2, quyết định pose ban đầu: nếu `/dock_status` cho biết robot đang trong dock thì phát `/initialpose` từ `goals.yaml`, ngược lại giữ ước lượng hiện có của AMCL. Bước 3, nếu robot đang trong dock thì gọi action `/undock` (tối đa 60 s theo giờ mô phỏng). Bước 4, với mỗi goal gọi `goToPose`, in phản hồi (khoảng cách còn lại, thời gian ước tính, số lần hồi phục) mỗi 3 s; goal thất bại thì xoá costmap và thử lại tối đa `max_retries` lần, quá `goal_timeout` thì huỷ. Khi tới nơi, node ghi sai số vị trí và hướng từ pose cuối cùng mà `bt_navigator` báo, dừng `pause_at_goal` giây để YOLO kịp quan sát, rồi sang goal kế. Bước 5, in bảng TỔNG KẾT và lưu `run.json` (quỹ đạo, đường Nav2 lập) cùng `summary.csv` vào `~/tb4_results/nav_<thời gian>/`. Nếu người dùng bấm Ctrl+C, node huỷ goal đang chạy để robot dừng và vẫn lưu kết quả của các goal đã đi. Các tham số điều chỉnh được nằm trong Bảng 8.




*Hình 7 – Lưu đồ của node* `waypoint_navigator.py`.

| Tham số | Mặc định | Ý nghĩa |
|---|---|---|
| `goals_file` | (rỗng) | file goal; rỗng = config/goals.yaml |
| `initial_pose_mode` | auto | auto: chỉ đặt pose khi robot trong dock; always; never |
| `undock` | true | tự undock khi robot đang trong dock |
| `pause_at_goal` | 2,0 s | thời gian dừng tại mỗi goal (giờ mô phỏng) |
| `goal_timeout` | 300 s | giới hạn thời gian cho một goal (giờ mô phỏng) |
| `max_retries` | 1 | số lần thử lại một goal thất bại (sau khi xoá costmap) |
| `look_around` | false | xoay 360° tại mỗi goal |
| `return_home`, `dock_at_end` | false | quay về chỗ sau undock; vào dock ở cuối |
| `loops` | 1 | số vòng đi qua các goal |

*Bảng 8 – Tham số của* `waypoint_navigator.py`.

```
ros2 launch tb4_lab04 navigation.launch.py   # Terminal 1: mô phỏng + AMCL + Nav2
ros2 run tb4_lab04 waypoint_navigator.py     # Terminal 2: robot tự đi G1, G2, G3
```

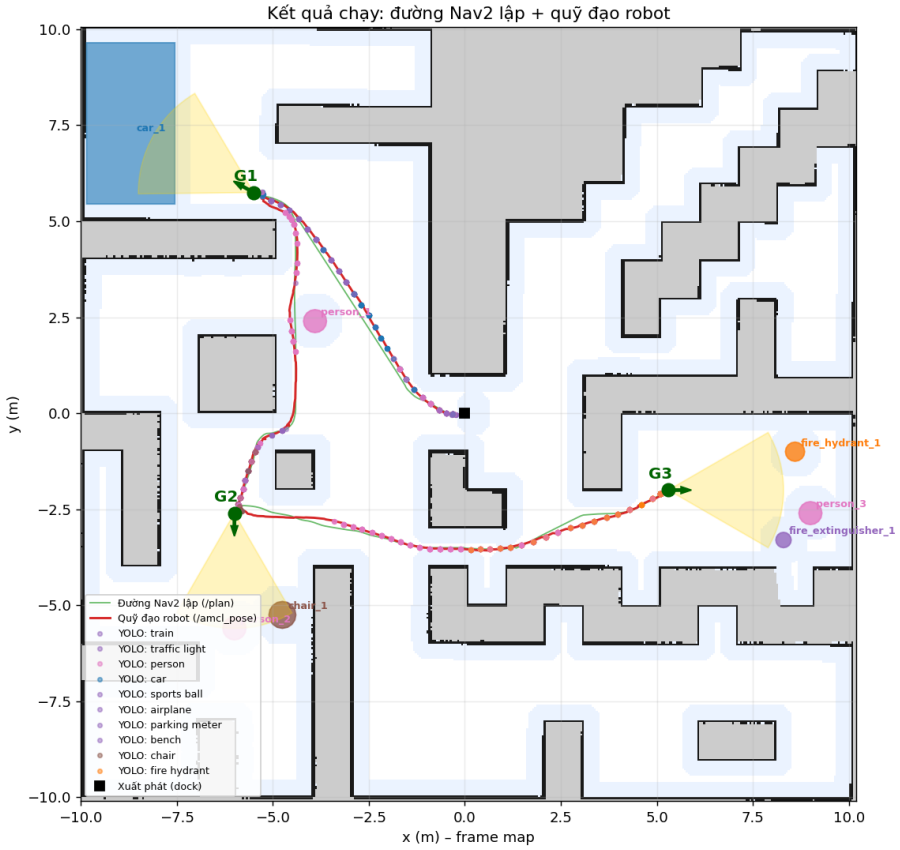
## 6.5. Kết quả điều hướng

Node điều hướng được chạy một lần, không điều khiển tay và không đặt goal bằng chuột trong suốt quá trình. Cả ba goal đều ở trạng thái SUCCEEDED ngay lần thử đầu (Bảng 9, Hình 8). Tổng cộng robot đi 30,5 m trong 118,2 s thời gian mô phỏng, tức tốc độ trung bình khoảng 0,26 m/s, bằng đúng vận tốc tối đa cấu hình cho DWB. Điều đó cho thấy robot gần như di chuyển liên tục với tốc độ tối đa, không mất thời gian đáng kể cho hành vi phục hồi. Quỹ đạo thực (đường đỏ) bám sát đường Nav2 lập (xanh lá), và đường đi ở cả ba chặng đều đi giữa hành lang, không men sát tường nhờ lớp vùng đệm.

| Goal | Kết quả | Thời gian (s) | Quãng đường (m) | Sai số vị trí (m) | Sai số hướng | Số lần thử |
|---|---|---|---|---|---|---|
| G1 | SUCCEEDED | 30,9 | 7,7 | 0,25 | 6° | 1 |
| G2 | SUCCEEDED | 39,5 | 11,1 | 0,24 | 14° | 1 |
| G3 | SUCCEEDED | 47,8 | 11,7 | 0,23 | 14° | 1 |
| **Tổng** | **3/3 đạt** | **118,2** | **30,5** | | | |

*Bảng 9 – Kết quả ba goal (thời gian theo giờ mô phỏng; sai số so với pose AMCL tại thời điểm kết thúc).*

Sai số vị trí 0,23–0,25 m nằm sát ngưỡng `xy_goal_tolerance` = 0,25 m: bộ kiểm tra goal kết thúc ngay khi robot vào trong ngưỡng, nên con số này phản ánh ngưỡng chấp nhận đã cấu hình chứ không phải độ chính xác tối đa của AMCL hay Nav2. Sai số hướng 6°–14° (0,10–0,24 rad) cũng nằm trong ngưỡng 0,25 rad. Cần lưu ý các sai số này được tính so với pose mà AMCL ước lượng, không phải so với pose thật trong Gazebo.



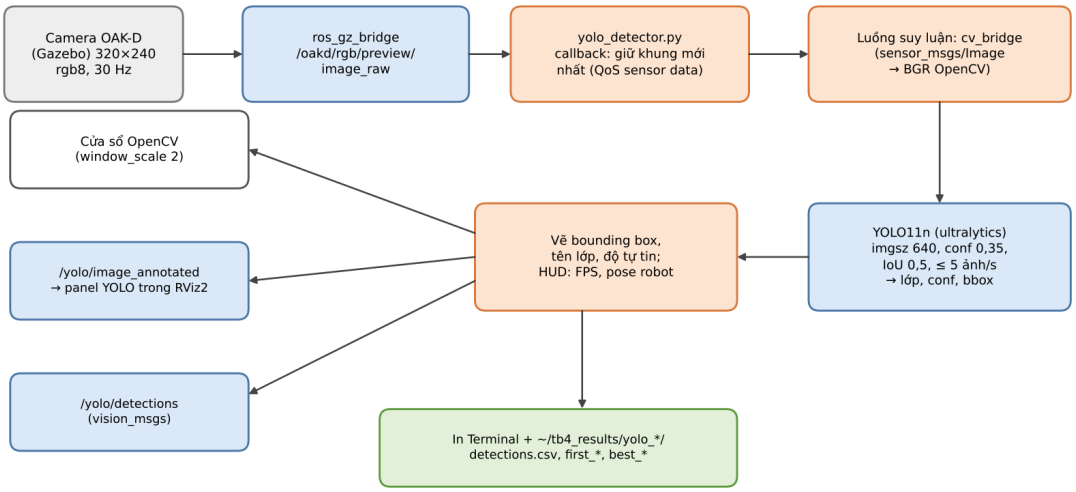

*Hình 8 – Đường Nav2 lập (xanh lá, `/plan`), quỹ đạo robot (đỏ, `/amcl_pose`) và vị trí robot khi YOLO phát hiện từng lớp (chấm màu).*

# 7. Nhận diện vật thể bằng YOLO (Phần 4)

## 7.1. Thiết kế node

Node `yolo_detector.py` (Hình 9) đăng ký topic `/oakd/rgb/preview/image_raw`. Hàm gọi lại chỉ giữ khung hình mới nhất; một luồng suy luận riêng lấy khung đó, chuyển `sensor_msgs/Image` sang ảnh BGR của OpenCV bằng `cv_bridge`, đưa qua mô hình YOLO và nhận về danh sách (lớp, độ tự tin, khung bao). Cách làm này giúp suy luận chậm không làm hàng đợi ảnh phình ra: khung cũ bị bỏ, luôn xử lý khung gần nhất, tối đa 5 ảnh mỗi giây để dành CPU cho Gazebo và Nav2. Node vẽ khung bao, tên lớp, độ tự tin và dòng trạng thái (tên mô hình, FPS, số vật thể, vị trí robot trong frame `map`) lên ảnh, rồi hiển thị trên cửa sổ OpenCV, phát lên `/yolo/image_annotated` để xem trên RViz2, phát `/yolo/detections` và in ra terminal (lớp mới xuất hiện được in ngay, danh sách đầy đủ in mỗi giây). Vị trí robot lấy từ `/amcl_pose`. Khi kết thúc, node lưu `detections.csv` cùng ảnh lần đầu thấy và ảnh tin cậy nhất của mỗi lớp vào `~/tb4_results/yolo_<thời gian>/`.

Quá trình lọc kết quả của YOLO dùng ngưỡng độ tự tin và loại bỏ khung trùng bằng NMS, trong đó hai khung A và B bị coi là trùng khi




*Hình 9 – Luồng xử lý của node* `yolo_detector.py`.

$$\text{IoU}(A,B) = \frac{|A \cap B|}{|A \cup B|} > 0{,}5 \tag{4}$$

## 7.2. Mô hình và tham số

Bài dùng YOLO11n của thư viện ultralytics, mô hình nhẹ nhất của họ YOLO11, đã huấn luyện sẵn trên bộ COCO (80 lớp) và chạy bằng PyTorch bản CPU. Ảnh camera mô phỏng chỉ có 320 × 240 điểm ảnh nên được phóng lên 640 khi suy luận để nhận vật nhỏ và xa tốt hơn (Bảng 10).

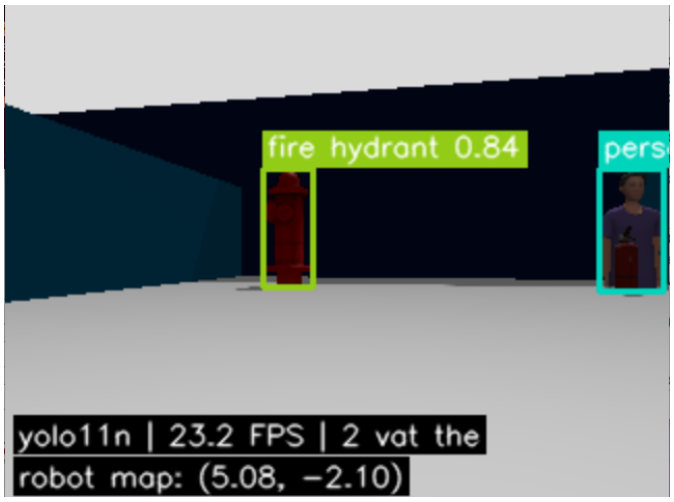
| Tham số | Giá trị | Ý nghĩa |
|---|---|---|
| `model` | yolo11n.pt | mô hình; có thể đổi yolov8n.pt hoặc YOLO-World (yolov8s-worldv2.pt) |
| `conf` | 0,35 | ngưỡng độ tự tin; ảnh nhỏ nên không đặt quá cao |
| `iou` | 0,5 | ngưỡng NMS |
| `imgsz` | 640 | kích thước ảnh khi suy luận |
| `max_rate` | 5 ảnh/s | giới hạn tốc độ để dành CPU cho Gazebo và Nav2 |
| `window_scale` | 2,0 | phóng cửa sổ OpenCV cho dễ nhìn |

*Bảng 10 – Tham số chính của* `yolo_detector.py` *(*`config/yolo.yaml`*).*

```
ros2 launch tb4_lab04 yolo.launch.py  # node YOLO + cửa sổ camera
ros2 launch tb4_lab04 yolo.launch.py yolo_model:=yolo11s.pt conf:=0.5   # model lớn, conf cao
```

## 7.3. Kết quả nhận diện

Trong lần chạy, robot vừa di chuyển qua ba goal vừa nhận diện vật thể (Hình 10). Bảng 11 là bảng tổng kết do node in ra khi kết thúc; cột "Số khung" đếm số khung hình có xuất hiện lớp đó (mỗi khung đếm một lần cho mỗi lớp), không phải số vật thể.




*Hình 10 – Cửa sổ nhận diện YOLO khi robot ở gần G3: trụ cứu hoả (fire hydrant, 0,84) và người (person) bị cắt ở mép ảnh.*

| Lớp | Số khung | Conf tối đa | Lần đầu (s) | Robot lúc thấy lần đầu (map) | Có trong mê cung? |
|---|---|---|---|---|---|
| person | 3484 | 0,91 | 57,5 | (−0,30; −0,02) | có (3 vật thể) |
| fire hydrant | 2587 | 0,88 | 162,1 | (0,15; −3,55) | có |
| car | 212 | 0,93 | 57,8 | (−0,31; −0,02) | có |
| chair | 84 | 0,86 | 128,0 | (−5,47; −1,01) | có |
| airplane | 388 | 0,94 | 63,1 | (−0,32; −0,02) | không |
| bench | 54 | 0,79 | 127,8 | (−5,40; −0,88) | không |
| traffic light | 14 | 0,75 | 53,0 | (0,01; −0,02) | không |
| sports ball | 6 | 0,62 | 63,1 | (−0,32; −0,02) | không |
| train | 1 | 0,55 | 52,2 | (0,01; −0,02) | không |
| parking meter | 1 | 0,51 | 102,7 | (−4,45; 4,93) | không |

*Bảng 11 – Tổng kết YOLO của lần chạy (thời gian theo giờ mô phỏng).*

Các lớp có thật trong mê cung (người, trụ cứu hoả, xe hơi, ghế) đều được nhận diện với độ tự tin cao nhất từ 0,86 đến 0,93. Số khung lớn của person và fire hydrant chủ yếu do hai vật thể này nằm trước camera khi robot dừng ở G3 nên nhiều khung gần như trùng nhau; con số không phản ánh số lượng vật thể. Vị trí robot lúc phát hiện được vẽ thành các chấm màu trên Hình 8, cho thấy từng lớp được nhìn thấy ở đoạn nào của đường đi.

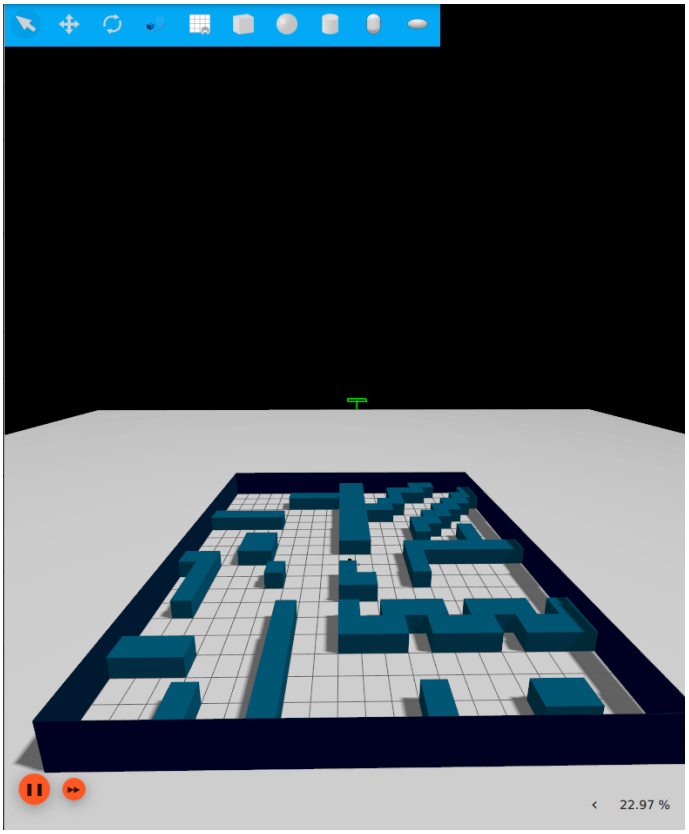
Bên cạnh đó có hai nhóm kết quả cần xem xét. Thứ nhất, sáu lớp không tồn tại trong mê cung vẫn xuất hiện: airplane (388 khung, độ tự tin tối đa 0,94), bench (54 khung) và bốn lớp còn lại chỉ vài khung. Với mô hình nhẹ trên ảnh mô phỏng 320 × 240 và ngưỡng 0,35, đây là các nhận nhầm; riêng airplane có độ tự tin cao và lặp lại nhiều khung nên nhiều khả năng là một vật thể hay bề mặt cố định bị nhận nhầm một cách có hệ thống. Thứ hai, bình chữa cháy không thuộc 80 lớp COCO nên YOLO11n không gọi được tên nó: trong Hình 10 khung "person" còn bao cả bình màu đỏ trước ngực. Muốn nhận đúng bình chữa cháy cần mô hình có lớp này, chẳng hạn YOLO-World với danh sách lớp tự đặt (package đã hỗ trợ) hoặc huấn luyện thêm. Để giảm nhận nhầm có thể nâng ngưỡng `conf` lên 0,5 hoặc giới hạn tham số `classes` vào các lớp cần quan tâm.

## 8. Nhận xét và hạn chế

Hệ thống đáp ứng các yêu cầu của bài: (1) mô phỏng chạy lại trong đúng world `maze` cùng bảy vật thể mới, bản đồ tuần trước được nạp và AMCL cùng Nav2 hoạt động; (2) ba goal được xác định và kiểm tra; (3) node điều hướng tự đưa robot đi qua G1 → G2 → G3 mà không điều khiển tay, cả ba goal đạt; (4) node YOLO chạy song song, hiển thị khung, tên lớp, độ tự tin và in ra terminal. Một số hạn chế cần nêu rõ.

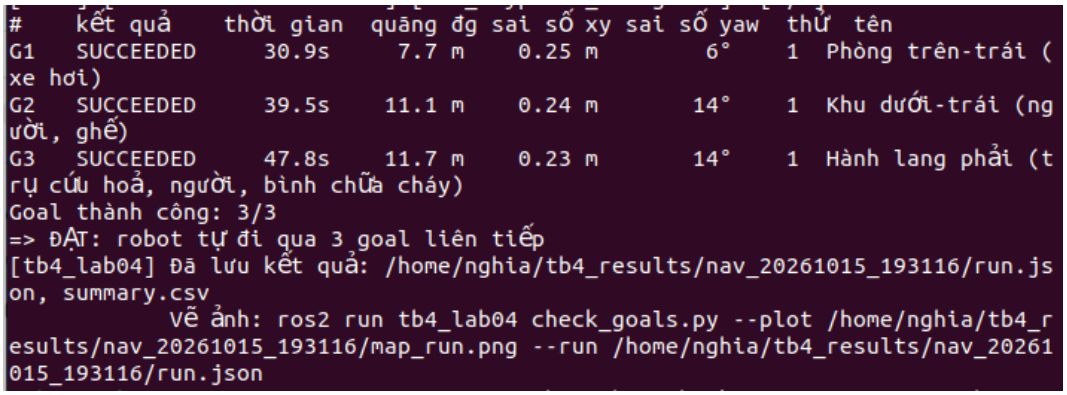
Thứ nhất, kết quả điều hướng lấy từ một lần chạy duy nhất nên chưa đánh giá được độ ổn định giữa các lần, và lần chạy này chưa đo số lần va chạm bằng `nav_monitor.py`; nhận xét "không va chạm" cần dựa vào video. Thứ hai, sai số tới đích được tính so với pose AMCL, không phải pose thật trong Gazebo, và bị chặn bởi ngưỡng 0,25 m. Thứ ba, tọa độ goal được chọn từ bản đồ và kiểm tra bằng script, không phải do robot đứng tại chỗ đo. Thứ tư, YOLO11n còn nhận nhầm một số lớp, không nhận được bình chữa cháy, và chạy trên CPU nên bị giới hạn 5 ảnh/s; hướng cải thiện là nâng ngưỡng conf, lọc lớp, dùng YOLO-World hoặc mô hình lớn hơn khi có GPU.

## 9. Hình ảnh minh hoạ kết quả




*Hình 11 – World* `maze` *trong Gazebo Fortress (ảnh chụp ở bài trước, trước khi thêm vật thể; cùng world và vị trí dock).*

Bản đồ kèm các goal ở Hình 5, đường Nav2 lập cùng quỹ đạo robot ở Hình 8, ảnh nhận diện YOLO ở Hình 10. Quá trình nạp bản đồ, định vị, robot tự đi qua từng goal và cửa sổ YOLO chạy song song được ghi lại trong video demo.




*Hình 12 – Terminal: bảng TỔNG KẾT của* `waypoint_navigator.py` *(3/3 goal đạt).*

## 10. Liên kết video demo và mã nguồn

**Video demo** (Google Drive, chế độ công khai):

[CHƯA CÓ LINK – điền vào docs/report_info.yaml rồi dựng lại báo cáo]

**Mã nguồn** (GitHub, chế độ Public):

[CHƯA CÓ LINK – điền vào docs/report_info.yaml rồi dựng lại báo cáo]